Motivation

- Leaky Integrate-and-Fire Neurons
    - work by integrating (accumulating) input currents over time to increase a "membrane potential" and "fire" (produce a spike) when this potential crosses a threshold, similar to how a leaky RC circuit charges. After firing, the potential resets. The "leaky" part refers to the fact that the membrane potential also gradually decays or "leaks" back to a resting value over time unless it is being driven by input
- The SCN idea
    - a population of LIF neurons only fire when it reduces the global representation error. This yields spare, irregular, yet coordinated spikes
    - The paper shows that the same network can implement a linear dynamical system (add a “slow” recurrent term).
    - Kalman filter for optimal state estimation through noisy observations
    - LQR for optimal control to a target z(t)
    - The control u is a linear readout of the spikes
    - -> Spiking LQG

Setup

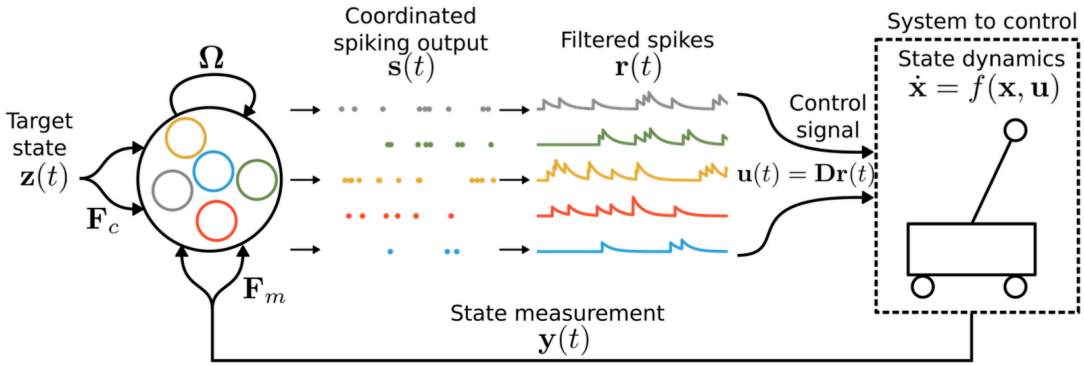

Fig. 1: Controlling dynamical systems with spiking neural networks. A recurrent spiking neural network (left) will emit spikes based on its inputs and connectivity, which are translated into a control signal to control some target system (right). $\mathbf{F}_c$ and $\mathbf{F}_m$ represent input weights, $\mathbf{\Omega}$ represent recurrent weights, and $\mathbf{D}$ represent read-out weights. By first defining the optimal control solution for the system, and then directly translating the resulting control parameters into these connection weights, we can generate a robust and population-wide coordinated spiking network that accurately controls identified systems.

- x: state
- y: incomplete measurement of the system state
- z: incomplete measurement of the target state
- u: control signal
- s: spiking signal emitted from a recurrent SNN, used to control the stater: filtered spikes

Emitted Spikes and Filtered Spikes

- s: spiking signal emitted from a recurrent SNN, used to control the state
  - $$s_i(t) \;=\; \sum_k \delta\big(t - t_i^{(k)}\big), \qquad s(t) = \big[s_1(t), \ldots, s_N(t)\big]^\top.$$
  - $$s_i[n] = \begin{cases} \frac{1}{\Delta t}, & \text{if neuron } i \text{ spikes in bin } n, \\ 0, & \text{otherwise,} \end{cases}$$ discrete:
  - s_i(t) is the sum of the spikes emitted from the neurons in time t.
    - Each $t_i^{(k)}$ is the k-th spike time of neuron i.
    - Integrating s_i over any interval counts spikes: $\int_{t_0}^{t_1} s_i(t)\,dt = \#\{\text{spikes of } i \text{ in } [t_0, t_1]\}$.
    - This is a **distribution** (generalized function), not an ordinary continuous signal. In their LIF network, a spike occurs when the voltage $v_i(t)$ hits its threshold T_i; then s_i contributes a delta at that instant.
- r: filtered spikes

$$\dot{r}(t) = -\lambda r(t) + s(t) \quad \Rightarrow \quad r = \big(e^{-\lambda t} H(t)\big) * s,$$

  - so each spike adds a unit jump to r that then decays with time constant $1/\lambda$. Closed-form control with spike ...
  - $$\dot{r}(t) \;=\; -\lambda\, r(t) + s(t) \quad \Longleftrightarrow \quad r(t) = \int_{-\infty}^{t} e^{-\lambda(t-\tau)}\, s(\tau)\, d\tau.$$
- Dirac impulse delta function
  - $$\delta(x) = \begin{cases} 0, & x \neq 0 \\ \infty, & x = 0 \end{cases} \quad \text{with} \quad \int_{-\infty}^{\infty} \delta(x) dx = 1.$$
  - it's not a real function, so approximated with limits: $\delta_a(x) = \frac{1}{|a|\sqrt{\pi}} e^{-(x/a)^2}$ a->0

Decoding matrix

- Assumption
  - the input signal estimate $\hat{x}$ can be linearly decoded from the spiking activity as $\hat{x} = Dr$, where D in R^(N x K) are known decoding weights (in this paper randomly drawn from the normal distribution and normalized)
  - spikes should only be emitted when this improves the coding error defined by the L2-norm (yielding a GREEDY SPIKING RULE): neuron i should spike iff $||x - Dr||_2^2 > ||x - Dr - D_i||_2^2$
- Definition

$$\hat{x}(t) \in \mathbb{R}^K \;=\; D\,r(t), \quad r(t) \in \mathbb{R}^N.$$

So each **column** $D_i \in \mathbb{R}^K$ says "how a spike from neuron i moves the estimate." Because a spike makes r_i jump by 1, it adds exactly D_i to $\hat{x}$. The paper also uses $D^\top$ as the **forward (encoding) weights** that inject the input x into voltages, which fixes the orientation: D is $K \times N$ (so $D^\top$ is $N \times K$).

In short: D **is the** $K \times N$ **dictionary that decodes the network's filtered spikes into the state estimate; its columns are the one-spike "basis vectors," and it induces both thresholds and the fast coordination term** $-D^\top D$**.**

- How to choose D
  - sample the columns of D randomly (Gaussian) and normalize them

    If you're building this:

    - pick $N \geq K$ (often $N \gg K$),
    - draw $D_i \sim \mathcal{N}(0, I_K)$, then normalize each D_i (column),
    - keep this D fixed; it defines the coding dictionary and all induced weights.
- Calculating Threshold
  - $||e||_2^2 \;>\; ||e - D_i||_2^2$
  - $||e||_2^2 \;>\; ||e||_2^2 - 2D_i^T e + ||D_i||_2^2$
  - $0 \;>\; 0 - 2D_i^T e + ||D_i||_2^2$
  - $D_i^T e \;>\; \frac{1}{2}||D_i||_2^2$
  - $v_i \;>\; \frac{1}{2}||D_i||_2^2$
  - so set the threshold T_i as such

Fast Recurrent Connection: Omega_f

- Target behavior we want
  - voltages should always equal "projection of the current error onto each decoder"
  - $v_j \;=\; D_j^T(x - \hat{x}) \;=\; D_j^T e$
- What happens after neuron i spikes
  - $\hat{x}$ becomes $\hat{x}' \;=\; \hat{x} + D_i$
  - e becomes $e' = x - \hat{x} - D_i \;=\; e - D_i$
  - the new correct voltage v_j becomes

    $v_j' \;=\; D_j^T e' \;=\; D_j^T e - D_j^T D_i \;=\; v_j - D^T D_{ji}$

- How to implement that instantly: choose the fast recurrent weight matrix
  - $\Omega_f = -D^T D$
  - No double-encoding: neurons whose decoders look like D_i(large $D_j{}^T D_i$) get their voltages pulled down the most, so they don't reflexively fire to add the same direction again.
  - Self-reset: the diagonal entry $-||D||^2$ knocks down v_i strongly after it spikes, preventing immediate re-firing unless the residual still demands it.

Voltage: Tracking a fully observed x

- LIF formula under full information
  - $\dot{v} = -\lambda v + D^T(\dot{x} + \lambda x) + \Omega_f s$
- Proof
  - $\dot{v} = D^T(\dot{x} - \dot{\hat{x}})$
  - $\dot{v} = D^T(\dot{x} - D\dot{r})$
  - $\dot{v} = D^T\dot{x} - D^T D\dot{r}$
  - $\dot{v} = D^T\dot{x} - D^T D(-\lambda r + s)$
  - $\dot{v} = D^T\dot{x} + D^T D\lambda r - D^T D s$
    - notice $v = D^T x - D^T D r$
    - giving $D^T D r = D^T x - v$
  - $\dot{v} = -\lambda v + D^T(\dot{x} + \lambda x) - D^T D s$
  - $\dot{v} = -\lambda v + D^T(\dot{x} + \lambda x) + \Omega_f s$

Voltage: Implementing a dynamical system

- What changed
  - Assume a linear dynamical system of the form:
    - $\dot{x} = Ax$
  - In addition, replace the external state x with the network's internal estimate:
    - $\hat{x} = Dr$
- New formula
  - $\dot{v} = -\lambda v + D^T(\dot{x} + \lambda x) + \Omega_f s$

- $\dot{v} = -\lambda v + D^T(Ax + \lambda x) + \Omega_f s$
- $\dot{v} = -\lambda v + D^T(A + \lambda I)Dr + \Omega_f s$
- $\dot{v} = -\lambda v + \Omega_s r + \Omega_f s$

- Giving rise to Ω_s
  - we defined $\Omega_s = D^T(A + \lambda I)D$

    where the derivation has resulted in an additional set of slow connections implementing the desired dynamics given by $\mathbf{\Omega}_s = \mathbf{D}^\top(\mathbf{A} + \lambda\mathbf{I})\mathbf{D}$. In essence, the slow connectivity reads out the internal estimate of $\mathbf{x}$ through $\mathbf{D}$. It then applies the dynamics of the linear dynamical system through $\mathbf{A} + \lambda\mathbf{I}$, and encodes the result back into the network using $\mathbf{D}^\top$. The network now keeps track of its own estimate on a fast time scale through the fast connections, and drifts this estimate according to the dynamics $\mathbf{A}$ through the slow recurrent connections.
  -

Voltage: Kalman Estimation

- Linear dynamical system
  - $\dot{x}(t) = Ax(t) + Bu(t) + \eta_d$
  - $y(t) = Cx(t) + \eta_n$
  - Goal: control x to some reference state z
- Kalman filter in continuous time is given by a dynamical system of the form
  - $\dot{\hat{x}} = A\hat{x} + Bu + K_f(\hat{y} - y)$
  - $\dot{\hat{x}} = ADr + Bu + K_f(CDr - y)$
  - The Kalman filter gain matrix is applied to an error between the observations of the dynamical system and the Kalman filter's own internal estimate, ˆy= Cˆx.
  - For fully identified systems Kf can be found by solving an algebraic Riccati equation.
  - We will assume that Kf is known
- New formula
  - $\dot{v} = -\lambda v + D^T(\dot{x} + \lambda x) + \Omega_f s$

- $\dot{v} = -\lambda v + D^T(ADr + Bu + K_f(CDr - y) + \lambda Dr) + \Omega_f s$
- $\dot{v} = -\lambda v + D^T(A + \lambda I)Dr + D^T Bu + D^T K_f CDr - D^T K_f y + \Omega_f s$
- $\dot{v} = -\lambda v + \Omega_s r + F_i u + \Omega_k r - F_k y + \Omega_f s$

- Definitions
  - old definitions
    - $\Omega_f = -D^T D$
      - fast recurrent connections
    - $\Omega_s = D^T(A + \lambda I)D$
      - slow recurrent connections
  - new definitions
    - $F_i = D^T B$
      - input control connections
    - $\Omega_k = D^T K_f CD$
      - recurrent Kalman filter connections
    - $F_k = D^T K_f$
      - feed-forward Kalman filter connections